

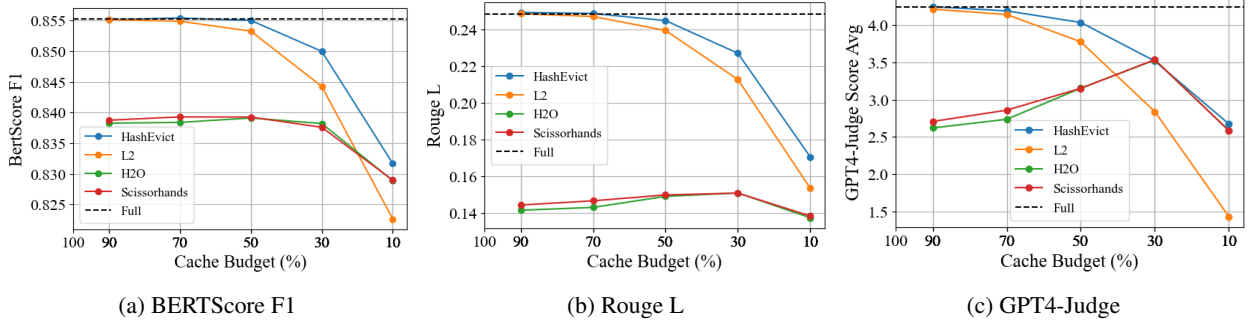


Figure 3: **MedQA Question Answering Performance.** We measure BertScore F1, Rouge-L, and GPT4-Judge for different cache budgets on a medical exam task. LSH outperforms L_2 for all three metrics for every budget, with a significantly higher performance for the 30% and 10% budgets.

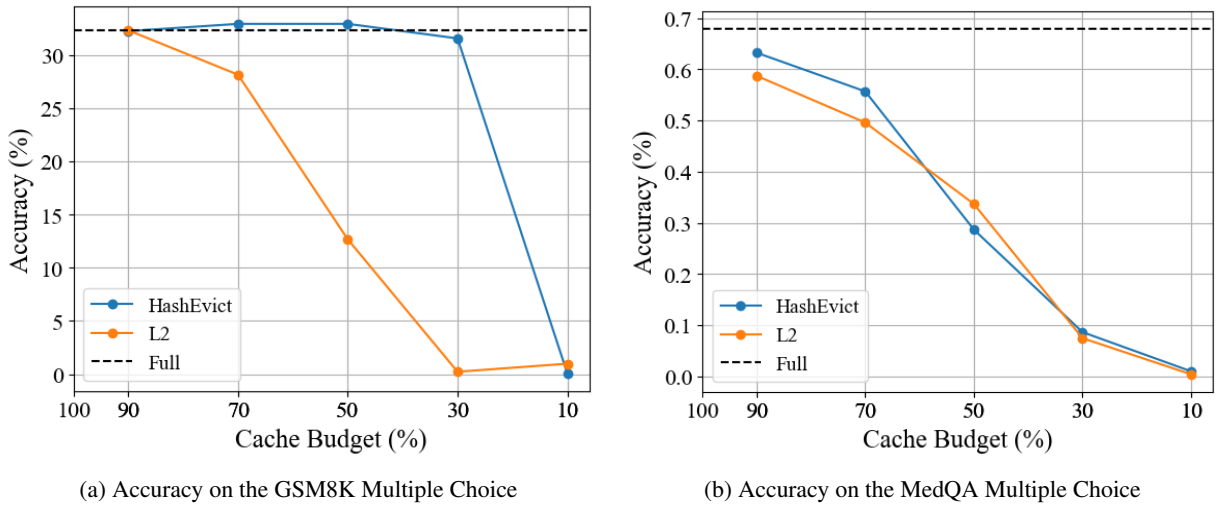


Figure 4: **Multiple Choice Tasks Performance.** On GSM8K, HASHEVICT outperforms the baseline full cache on GSM8K at 70% and 50% cache budgets and significantly outperforms L_2 at 70%, 50%, and 30%. HASHEVICT performs on par with L_2 overall on MedQA with higher performance at 90% (near uncompressed performance) and 70% budget and slightly lower performance at 50% budget.

MedQA Multiple Choice Per Figure 4b, the MedQA multiple choice experiment, HASHEVICT offers better performance than L_2 eviction for all tested cache budgets except for 50%. Performance between both methods is highly similar at lower budgets.

4.3 Long-Context Retrieval

To evaluate HASHEVICT’s ability to retain and retrieve important pieces of information from long contexts, we used the Needle-in-a-Haystack and Common Words tasks from Hsieh et al. [2024] with 4K context length. These tests benchmark the ability of a compression strategy to retain important tokens inside the KV cache within a large, complex stream of context.

Needle-in-a-Haystack In the Needle-in-a-Haystack task, the model must extract specific information buried within a large body of text. As illustrated in Figure 5b, HASHEVICT slightly outperforms L_2 at every cache budget except for 90%, and both methods see a sharp drop in the ability to recall the “needle” (a small, targeted piece of context) after the cache budget drops to 50% and lower. HASHEVICT outperforms L_2 for these smaller cache sizes.

Common Words In the Common Words task, the model must identify the most frequent words from a long list. Figure 5a demonstrates that HASHEVICT performs on par with L_2 eviction in general and slightly better at 30%, 50%, and 90% cache budget. Both methods outperform the full cache model at 90% cache size, indicating that some cache

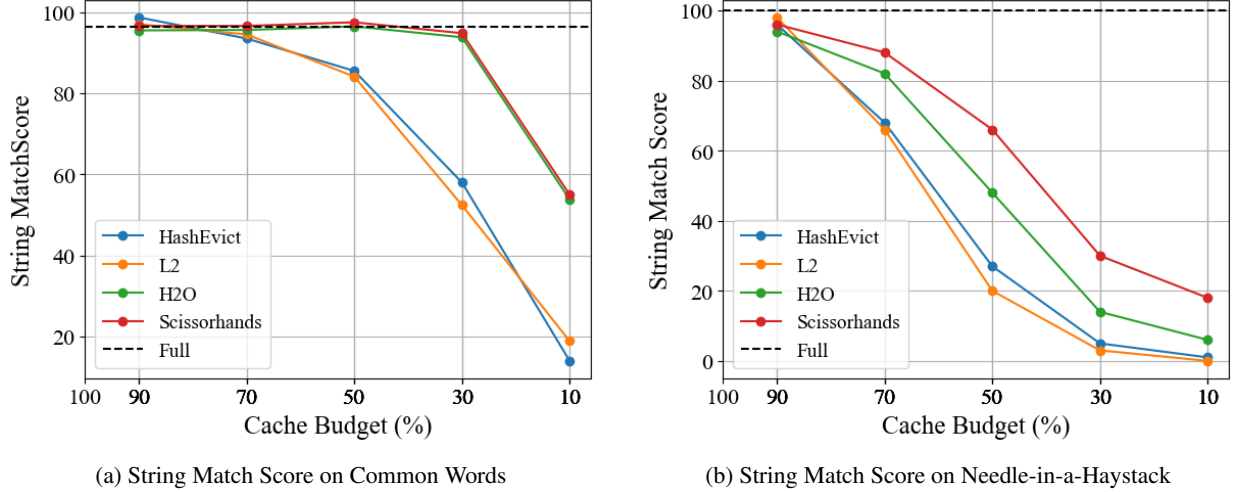


Figure 5: **Long-Context Tasks.** We measure string-matching scores for two long-context retrieval tasks. HASHEVICT performs on par with L_2 on the Common Words task with slightly higher performance at a 30% cache budget and slightly lower performance at a 10% budget. For the Needle-in-a-Haystack task, HASHEVICT performs on par with L_2 with slightly higher performance at a 50% cache budget.

compression can actually increase performance. Neither method experienced a significant drop in performance until the cache budget was reduced to 30%.

4.4 Long Text Summarization

To evaluate HASHEVICT’s ability to handle exceptionally long context lengths, we incorporated the MultiNews and GovReport summarizations tasks from LongBench Bai et al. [2023]. We tested both tasks using the Llama3.1-8B-Instruct model and used context size of 16K tokens.

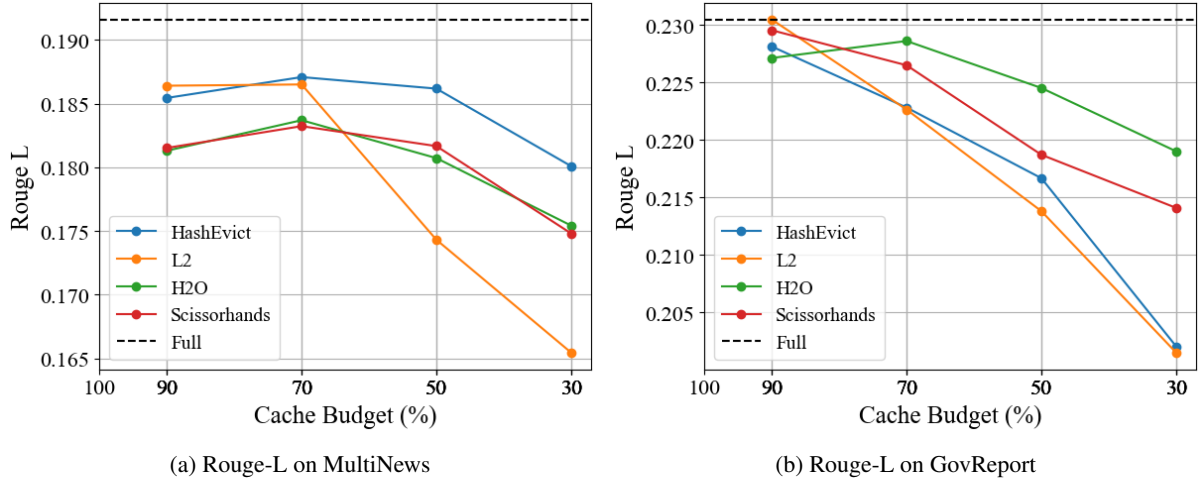


Figure 6: **LongBench Summarization Tasks** We measure Rouge-L for two long text summarization tasks. HASHEVICT outperforms all baseline methods on MultiNews at 30 - 70% cache budget. HASHEVICT performs better than L_2 on GovReport at 50% cache budget similarly at 30% and 70%.

MultiNews The MultiNews dataset contains clusters of 2-10 news articles discussing the same event or topic. The model is asked to provide a one-page summary of the articles. HASHEVICT outperforms all baselines in the MultiNews summarization task at 30-70% cache budget. At 90% cache budget, HASHEVICT still outperforms H_2O and Scissorhands while being slightly lower than L_2 .

GovReport The GovReport dataset contains reports spanning a wide variety of national policy issues from the U.S. Government. The model is asked to produce a one-page summary of the reports. HASHEVICT performs on par with and sometimes slightly better than L_2 at 30-70% cache budget, while not as well as H₂O or Scissorhands.

4.5 Throughput

To evaluate the speed of HASHEVICT and baseline methods, we measured the decoding and prefilling speed during the MultiNews evaluation. Because the length of answers generated by each eviction strategy generates can be different, we report decoding and prefilling speed in tokens per second instead of elapsed time.

Table 1: **Throughput on LongBench MultiNews Summarization Task** HASHEVICT method is as fast as L_2 and faster than other baselines at both prefilling and decoding, even without low-level optimizations (i.e., expressing our hash tables in true binary bits). At the prefill stage, HASHEVICT is 1.5x as fast as H₂O and Scissorhands and 17x as fast compared to FastGen.

Cache Budget (%) / Fastgen Attn Recovery Frac (%)	Strategy	Rouge L	Decode Toks Per Sec	Prefill Tokes Per Sec
30	HASHEVICT	0.180	22.880	20293.524
	L_2	0.165	23.981	20628.160
	H ₂ O	0.175	21.555	13025.776
	Scissorhands	0.175	21.448	13004.254
50	HASHEVICT	0.186	22.846	20459.961
	L_2	0.174	16.013	15851.952
	H ₂ O	0.181	21.973	13969.985
	Scissorhands	0.182	20.978	13549.967
70	HASHEVICT	0.187	22.914	21002.334
	L_2	0.187	24.305	21303.763
	H ₂ O	0.184	21.793	14050.521
	Scissorhands	0.183	21.705	13954.693
90	HASHEVICT	0.185	22.873	21229.230
	L_2	0.186	24.010	21305.693
	H ₂ O	0.181	21.665	14007.697
	Scissorhands	0.182	21.411	14025.440
100	Full	0.192	16.071	16573.492
70	Fastgen	0.129	12.752	1171.069
75		0.174	12.291	1157.987
80		0.184	11.850	1142.679
85		0.183	11.658	1164.689

4.6 Memory Usage

Table 2 compares the memory usage of the KV cache and relevant data structures of L_2 and HASHEVICT on the GSM8K and MedQA question answering experiments. HASHEVICT maintains $\mathcal{H}$, a binary hash matrix of the attention keys in memory and, therefore, has slightly higher memory usage than L_2 eviction. Our implementation uses 8 bits for binary values instead of 1 bit. Using 1-bit binary numbers would reduce the memory overhead of HASHEVICT by a factor of 8 and narrow the difference in memory usage between HASHEVICT and L_2 .

4.7 Ablation on Hash Dimension

To determine the effect of the hash dimension, we conducted an ablation study using the GSM8K free response dataset. Fixing the cache budget to 50%, we tested hash dimensions of 4, 8, 16, 32 and 64 bits. The choice of hash dimension does not significantly impact performance. In fact, 8 bits performed the best, but not noticeably better than higher dimensions. This demonstrates that HASHEVICT does not require a high hashing dimension and can be executed with minimal storage overhead. When using 8 bits, the storage overhead is 1 byte $\times$ cache size. For example, in a Llama3